

Authentication Protocols

IS 2204 - Information Systems Security

Dilanga Harshani

University of Colombo School of Computing

July 27, 2026

Where You Are

- Introduction to Information Security and CIA Triad
- Symmetric and Asymmetric Cryptography
- Email and Document Security
- Transport Layer Security
- Payment Security and Blockchain
- Network Security Protocols
- Malicious Code and Defences
- **Authentication Protocols**

Today's Roadmap

- **Foundations**

What are we actually proving, and how?

- **Classical Authentication**

Passwords, one-time codes, Kerberos

- **Federated Identity**

One login across many services

- **Passwordless**

Authentication without a password at all

Identification, Authentication, Authorization

- **Identification**

Who does this user claim to be?

- **Authentication**

Can they prove who they claim to be?

- **Authorization**

Given that identity, what are they allowed to do?

- Three distinct questions, often conflated in casual conversations

Means of Authentication

- **Something you KNOW** : Password, PIN, Secret answer
- **Something you HAVE** : Smart card, Hardware token, Phone
- **Something you ARE** : Fingerprint, Iris pattern, Face
- Combining factors from different categories is what makes multi-factor authentication meaningfully stronger

Two-Factor Authentication (2FA)

- Combines two factors, typically something you know plus something you have
 - In Online banking, a password (something you know) is followed by an OTP sent to a registered phone (something you have)
- Meaningfully stronger than a single factor, but only as strong as the weaker of the two chosen factors

Multi-Factor Authentication (MFA)

- Combines two or more factors, potentially spanning all three categories
 - Enterprise VPN access requiring a password, a hardware token, and a fingerprint scan
- Stronger again in principle, but adds real friction and recovery becomes more complex as more factors are added

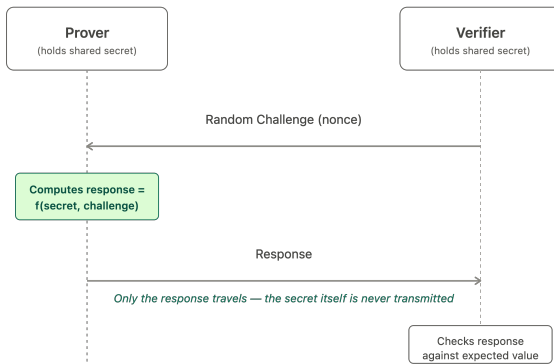
Authentication Based on a Secret

- **Direct Presentation** : Reveal the secret itself (typing a password)
- **Challenge-Response** : Prove knowledge of the secret without revealing it directly
- **Implicit (Cryptographic)** : Prove possession of a key through a cryptographic operation
- **Zero-knowledge** : Prove knowledge of the secret while revealing nothing about it

Direct Presentation

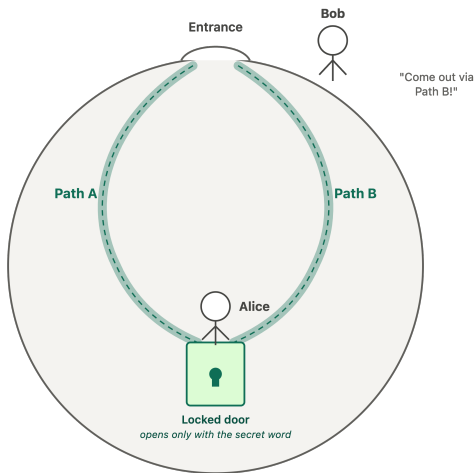
- The secret itself is transmitted directly to the verifier
- PAP (Password Authentication Protocol) sends credentials as plain text
- Offers no protection at all against a network eavesdropper

Challenge-Response Authentication



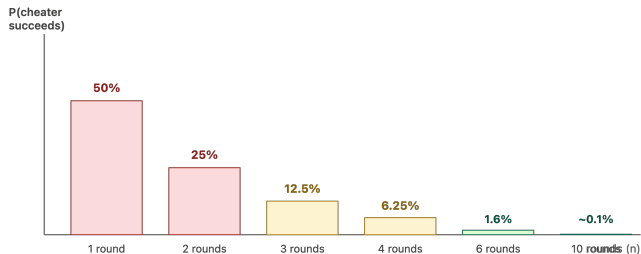
- The verifier issues a random challenge
- The prover computes a response derived from the secret and challenge
- Only the response is transmitted, the secret never leaves the prover

Zero-Knowledge Proof



- Proves knowledge of a secret, revealing nothing about the secret itself

Zero-Knowledge Proof



- Repeating the proof many times makes successful cheating vanishingly unlikely

Password Weaknesses

- Reused across multiple applications and services
- Vulnerable to guessing, interception, and offline cracking once a hash is obtained
- Tools such as **hashcat** are built specifically for recovering passwords from stolen hashes
- The 2011 Sony PlayStation Network breach exposed over 77 million password hashes
- The 2012 LinkedIn breach exposed over 100 million unsalted password hashes, cracked en masse within days

One-Time Passwords

- A password that is used once and never again, therefore intercepting it grants no future access
- Directly addresses the weaknesses of a static, reused password

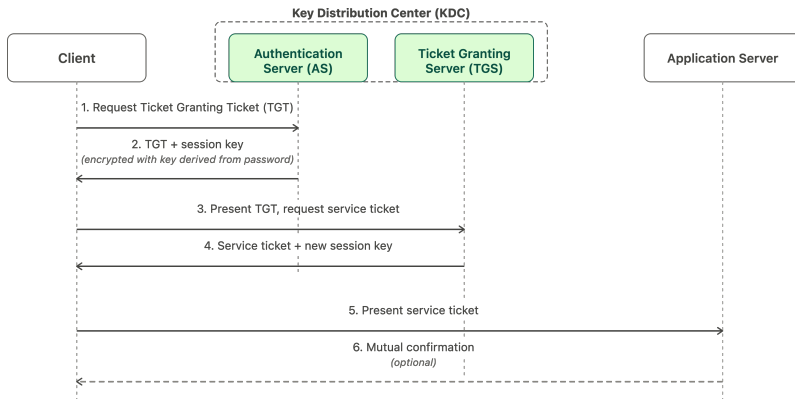
One-Time Password Mechanisms & Their Weaknesses

Mechanism	How It Works	Weakness
S/Key	Password generated by repeatedly hashing a seed	Seed compromise, or exhausting the hash sequence, ends its usefulness
CHAP	Challenge-response used during network authentication	Still ultimately reliant on a shared secret, weak if that secret is weak
SecurID	Hardware token generating a new code at fixed intervals	Token loss, or a breach of the seed database, compromises all associated tokens
Mobile OTP	Code delivered to a registered device	Vulnerable to SIM-swapping and SMS interception

Kerberos

- A trusted third-party authentication service for distributed environments
- Issues time limited tickets so a user does not need to re-authenticate to every service individually
- Adopted as the default authentication protocol in Microsoft Active Directory

How Kerberos Works



How Kerberos Works

- Client authenticates once with the **Authentication Server (AS)**, receiving a **Ticket Granting Ticket (TGT)**
- The TGT is presented to the **Ticket Granting Server (TGS)** to request a service ticket, without re-entering a password
- The service ticket is presented directly to the target application server to gain access

Limitations of Kerberos

- Still ultimately dependent on the strength of the underlying password
- Ticket lifetime is a genuine trade-off. Too short is impractical, too long increases exposure
- Requires reasonably synchronized clocks across all participating systems
- The Key Distribution Center (AS + TGS) is a single point of failure if compromised or unavailable
- Requires manual pre-arrangement between all parties involved

Authentication with Asymmetric Keys

- Proves identity by demonstrating possession of a private key, without any shared secret needing prior arrangement
- Raises a familiar question, “How does the verifier know a given public key genuinely belongs to the claimed party?”

Why Single Sign-On?

- Users hold identities across many independent services
- Maintaining a distinct password per service does not scale for the user
- Single Sign-On (SSO) authenticates once and grants access across multiple trusted services

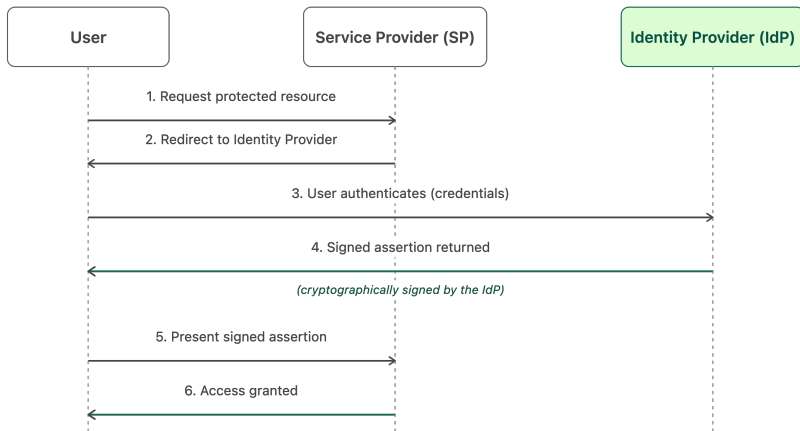
Identity Federation

- An agreement between separate organisations to trust each other's identity assertions
- Allows a user authenticated by one organisation to be recognised by another, without a separate account at each
- The technical basis for Single Sign-On across organisational boundaries

Security Assertion Markup Language (SAML)

- An XML-based standard for exchanging authentication and authorization data between an identity provider (IdP) and a service provider (SP)
- A SAML assertion conveys the confirmation of authentication, user attributes, and optionally an authorization decision

How SAML Works



How SAML Works

- User attempts to access a service provider
- The service provider redirects the user's browser to their identity provider
- The user authenticates directly with the identity provider
- The identity provider sends the signed assertion back to the browser, which then relays it to the service provider
- The signature from the identity provider lets the SP trust the assertion

OAuth and OpenID Connect

- **OAuth** is an authorization framework that grants an application limited access to a resource without sharing credentials
- **OpenID Connect** is an authentication layer built on top of OAuth, confirming who the user actually is
- “Sign in with Google” buttons use OpenID Connect specifically, not plain OAuth

SAML vs OpenID Connect

- SAML is common in legacy enterprise systems and academic / institutional federations
- OpenID Connect is common in modern applications, APIs, and social login

Moving Towards Passwordless

Approach	Weakness
Password alone	Phishable, reusable, breach-prone
Two-Factor (2FA)	Only as strong as its weaker factor, SMS OTP interceptable
Multi-Factor (MFA)	Real added friction, recovery often still falls back to a password
Federated Identity (SSO)	Fewer passwords to manage, but a compromised identity provider grants access everywhere

- Every approach so far still keeps a secret somewhere in the loop, which remains as the weakest link even with MFA and federation layered on top
- The industry's current direction is to remove the typed secret from the process entirely

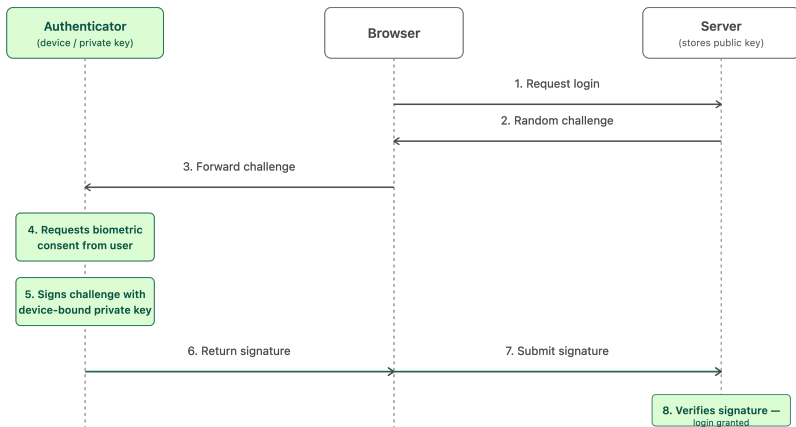
WebAuthn

- The Web Authentication API, a W3C and FIDO Alliance standard
- Uses a public / private key pair in place of a password
- The server stores only the public key, there is nothing secret worth stealing from it
- Removing the typed secret also removes most phishing risk

Passkeys

- WebAuthn credentials that are backed up and synced, commonly to a phone
- Effectively replaces a dedicated hardware security key with a device the user already carries
- Adopted at scale from 2022 onward by Apple, Google, and Microsoft as a unified passwordless standard

How Passkey Authentication Works



How Passkey Authentication Works

- Server issues a unique challenge tied to its own domain
- The authenticator requests biometric consent from the user
- The authenticator signs the challenge and returns it
- The server verifies the signature against the stored public key

Passwords vs Passkeys

Dimension	Password	Passkey
Secret transmitted?	Yes, typed and sent on every login	No, private key never leaves the device
Phishing risk	High, can be typed into a fake site	Low, challenge is bound to the real domain
Server breach impact	High, hashes can be cracked offline	Low, only a public key is stored
Reuse across sites	Common in practice	Not possible by design
Recovery method	Reset via email / security questions	Synced via cloud account or backup device

Human Verification : CAPTCHA

- A different question entirely, not “who are you,” but “are you human at all?”
- CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) uses challenges designed to be easy for humans, hard for automated bots
- Modern versions (e.g. reCAPTCHA v3) shift toward passive behavioural analysis rather than explicit visual puzzles

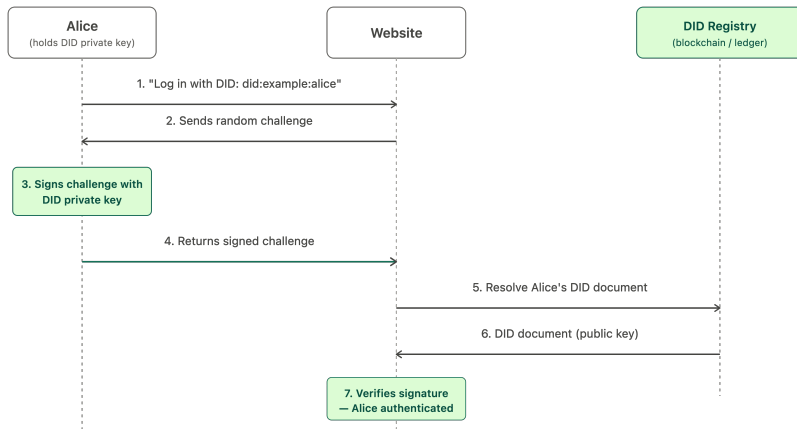
Emerging : Decentralized Identity

- **Decentralized Identifiers (DIDs)** : A cryptographically verifiable identifier owned by the user, with no central issuing authority
- **Verifiable Credentials** : Tamper-proof digital documents signed by an issuer and linked to a DID
- A preview of where identity may move next, not yet mainstream practice

Decentralized Identifiers (DIDs)

- A globally unique identifier created and owned by the user, not issued by any central authority
- Verified cryptographically, no central database required

Decentralized Identifiers (DIDs)



Verifiable Credentials (VCs)

- A tamper-proof digital version of a real-world credential (e.g. a licence)
- Contains a claim, the issuer's digital signature, and a link to the holder's DID
- Can be checked by any verifier without contacting the original issuer directly
- Together with DIDs, forms the basis of “self-sovereign identity”

So What's Actually Different?

Mechanism	Third party?	Why it's trusted	What's proven
Kerberos	Yes (KDC)	Pre-registered, same trusted realm	Possession of a ticket from a party already registered with
SAML, OAuth, OIDC	Yes (IdP)	Independent organisations agreeing to trust each other	The IdP vouches for you to an SP that has never met you
Passkeys	None	Direct device-to-service, nothing to trust in between	Possession of a private key shared with no one
DID	None	Self-issued, self-verified cryptographically	Possession of a key tied to an identifier you created yourself

Summary

- **Foundations** : Identification, authentication, and authorization are distinct questions; Factors combine into 2FA and MFA
- **Classical Authentication** : Secrets can be presented directly, challenged, or proven with zero knowledge; OTP and Kerberos each trade convenience against exposure
- **Federated Identity** : Identity federation let SAML and OAuth/OpenID Connect serve one identity across many services
- **Passwordless** : WebAuthn and passkeys remove the typed secret that every earlier approach still relied on
- **Emerging** : DIDs and Verifiable Credentials point toward user-owned identity, with no central authority at all